

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Sydney pursuant to Part VB of the Copyright Act 1968 (**the Act**). The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

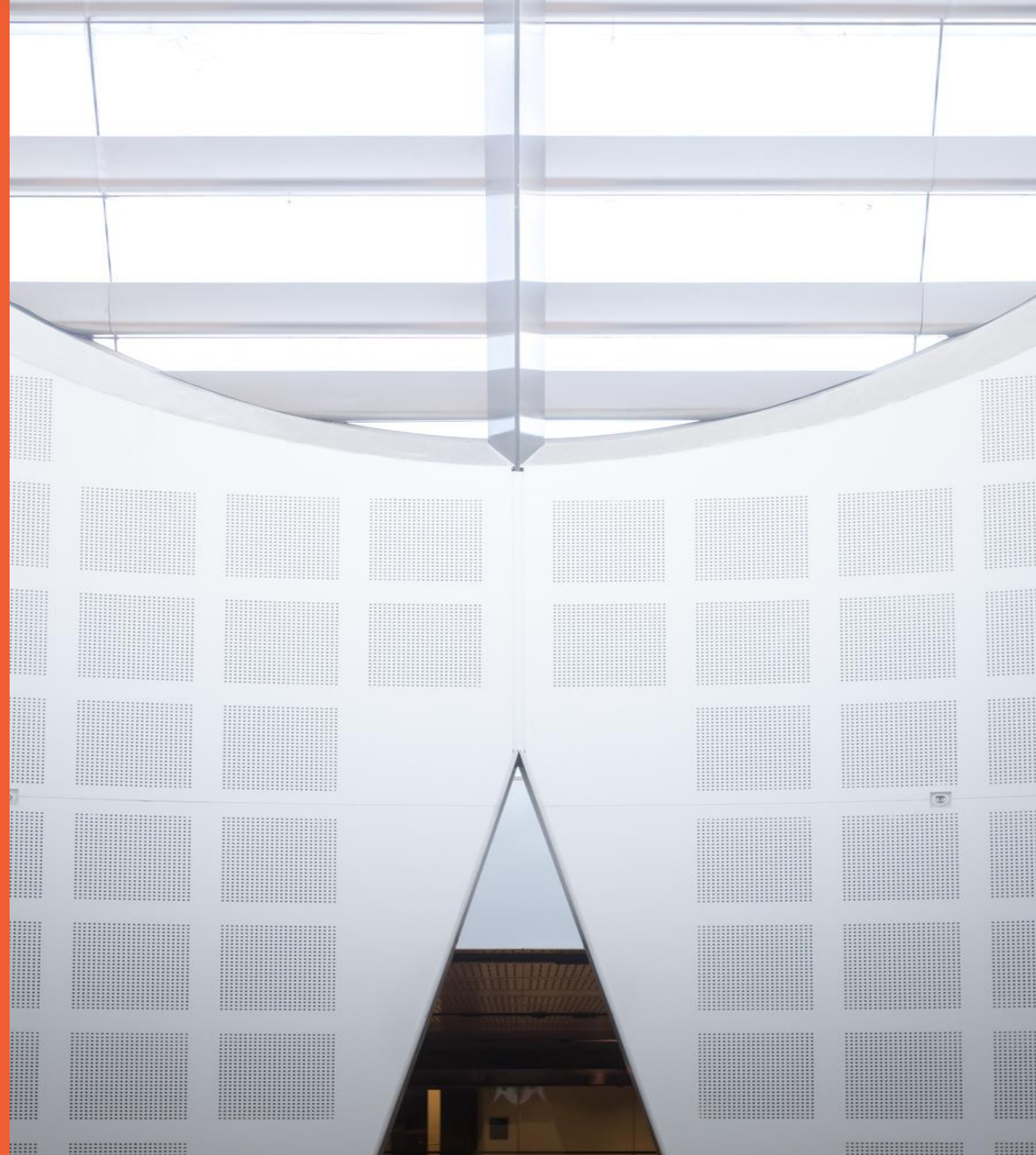
Do not remove this notice.

COMPx270: Randomised and
Advanced Algorithms
Lecture 10: Linear Programming
and Randomised Rounding

Clément Canonne
School of Computer Science



THE UNIVERSITY OF
SYDNEY



This week: Linear Programming, and Randomised Rounding

Maximize a linear function subject to linear inequality constraints on variables $x_1, \dots, x_n$ of interest.

Linear Programming

Maximize a linear function subject to linear inequality constraints on variables $x_1, \dots, x_n$ of interest.

$$\begin{aligned} &\text{maximise } \sum_{i=1}^n c_i x_i \\ &\text{subject to} \\ &\quad \sum_{i=1}^n A_{ji} x_i \leq b_j, \quad 1 \leq j \leq m \\ &\quad x_i \geq 0, \quad 1 \leq i \leq n \end{aligned}$$

Linear Programming

Example: Max Flow!

$$\text{maximise } \sum_{i=1}^n c_i x_i$$

subject to

$$\sum_{i=1}^n A_{ji} x_i \leq b_j, \quad 1 \leq j \leq m$$

$$x_i \geq 0, \quad 1 \leq i \leq n$$

Linear Programming

- Everything in **P** can be phrased as an LP
- LPs can be solved in **polynomial time** (in the size of the input)
- LPs can be solved **efficiently** (in practice)
- We're not going to see **any of that**

$$\text{maximise } \sum_{i=1}^n c_i x_i$$

subject to

$$\sum_{i=1}^n A_{ji} x_i \leq b_j, \quad 1 \leq j \leq m$$
$$x_i \geq 0, \quad 1 \leq i \leq n$$

Linear Programming

Use them to solve problems either exactly or **approximately**.

$$\text{maximise } \sum_{i=1}^n c_i x_i$$

subject to

$$\sum_{i=1}^n A_{ji} x_i \leq b_j, \quad 1 \leq j \leq m$$

$$x_i \geq 0, \quad 1 \leq i \leq n$$

$$\alpha \cdot \text{OPT}(I) \leq \text{VALUE}(S) \leq \text{OPT}(I)$$

Integer Linear Programming



More power!

But don't know how to solve them in general (efficiently)

Integer Linear Programming: st-Min-CUT

Integer Linear Programming: st-Min-CUT

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V$$

LP Relaxation: st-Min-CUT

$$\begin{aligned} & \text{maximise} \quad - \sum_{e \in E} c_e x_e \\ & \text{subject to} \\ & \quad y_s = 0 \\ & \quad y_t = 1 \\ & \quad y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & \quad x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V \end{aligned}$$

$$\begin{aligned} & \text{maximise} \quad - \sum_{e \in E} c_e x_e \\ & \text{subject to} \\ & \quad y_s = 0 \\ & \quad y_t = 1 \\ & \quad y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & \quad x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V \end{aligned}$$

LP Relaxation: st-Min-CUT

Fact 45.1. Let OPT_{ILP} be the optimal value of a solution to an ILP (maximisation problem), and OPT_{LP} be the optimal value of a solution to its LP relaxation. Then

$$\text{OPT}_{\text{ILP}} \leq \text{OPT}_{\text{LP}} .$$

(For a minimisation problem, the inequality is reversed.)

$$\begin{aligned} & \text{maximise} && - \sum_{e \in E} c_e x_e \\ & \text{subject to} && \\ & && y_s = 0 \\ & && y_t = 1 \\ & && y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & && x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V \end{aligned}$$

$$\begin{aligned} & \text{maximise} && - \sum_{e \in E} c_e x_e \\ & \text{subject to} && \\ & && y_s = 0 \\ & && y_t = 1 \\ & && y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & && x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V \end{aligned}$$

LP Relaxation: st-Min-CUT

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V$$

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V$$

LP Rounding!

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V$$

$$\text{maximise } - \sum_{e \in E} c_e x_e$$

subject to

$$y_s = 0$$

$$y_t = 1$$

$$y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E$$

$$x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V$$

LP Rounding: st-Min-CUT

$$\begin{aligned} & \text{maximise} && - \sum_{e \in E} c_e x_e \\ & \text{subject to} && \\ & && y_s = 0 \\ & && y_t = 1 \\ & && y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & && x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V \end{aligned}$$

$$\begin{aligned} & \text{maximise} && - \sum_{e \in E} c_e x_e \\ & \text{subject to} && \\ & && y_s = 0 \\ & && y_t = 1 \\ & && y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & && x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V \end{aligned}$$

-
- 1: Pick τ in $(0, 1)$ uniformly at random.
 - 2: **for all** $v \in V$ **do**
 - 3: Set $y_v = 1$ if $y_v^* > \tau$, 0 otherwise
 - 4: **return** y .
-

LP Rounding: st-Min-CUT

$$\begin{aligned} & \text{maximise} \quad - \sum_{e \in E} c_e x_e \\ & \text{subject to} \\ & \quad y_s = 0 \\ & \quad y_t = 1 \\ & \quad y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & \quad x_e, y_v \in \{0, 1\} \quad \forall e \in E, v \in V \end{aligned}$$

$$\begin{aligned} & \text{maximise} \quad - \sum_{e \in E} c_e x_e \\ & \text{subject to} \\ & \quad y_s = 0 \\ & \quad y_t = 1 \\ & \quad y_v \leq y_u + x_e, \quad \forall e = (u, v) \in E \\ & \quad x_e, y_v \in [0, 1] \quad \forall e \in E, v \in V \end{aligned}$$

LP and ILP in practice

- <https://au.mathworks.com/help/optim/ug/linprog.html>
- <https://au.mathworks.com/help/optim/ug/intlinprog.html>
- <https://reference.wolfram.com/language/ref/LinearProgramming.html>
- <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>
- <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>
- [...]

ILP+LP Relaxation+Rounding: Max-SAT

~~ILP+LP Relaxation+Rounding: Max-SAT~~

Theorem 47. *The “obvious” randomised algorithm which sets each variable x_i independently and uniformly at random gives, in expectation, a $\frac{1}{2}$ -approximation for MAX-SAT.*

~~ILP+LP Relaxation+Rounding: Max-SAT~~

Theorem 47. *The “obvious” randomised algorithm which sets each variable x_i independently and uniformly at random gives, in expectation, a $\frac{1}{2}$ -approximation for MAX-SAT.*

~~ILP+LP Relaxation+Rounding: Max-SAT~~

Theorem 47. *The “obvious” randomised algorithm which sets each variable x_i independently and uniformly at random gives, in expectation, a $\frac{1}{2}$ -approximation for MAX-SAT.*

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i:x_i \in C_j} y_i + \sum_{i:\neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\}$$

$$\forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\}$$

$$\forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\} \quad \forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\} \quad \forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\}$$

$$\forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\}$$

$$\forall 1 \leq j \leq m$$

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1$$

$$\forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1$$

$$\forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\} \quad \forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\} \quad \forall 1 \leq j \leq m$$

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1 \quad \forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1 \quad \forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

Input: Instance $\phi = (C_1, \dots, C_m)$ of MAX-SAT on n variables

- 1: Solve the LP relaxation (Fig. 16), getting solution (y^*, z^*) .
 - 2: **for all** $1 \leq i \leq n$ **do**
 - 3: Set $x_i = 1$ with probability y_i^* , independently of others.
 - 4: **return** x .
-

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$y_i \in \{0, 1\} \quad \forall 1 \leq i \leq n$$

$$z_j \in \{0, 1\} \quad \forall 1 \leq j \leq m$$

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1 \quad \forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1 \quad \forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

Theorem 48. *The randomised rounding given in Algorithm 20 gives, in expectation, a $(1 - \frac{1}{e})$ -approximation for MAX-SAT.*

Detour: the AM-GM inequality

Fact (AM–GM). For any $a_1, a_2, \dots, a_k \geq 0$,

$$\sqrt[k]{a_1 a_2 \dots a_k} \leq \frac{a_1 + a_2 + \dots + a_k}{k}$$

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1$$

$$\forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1$$

$$\forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1$$

$$\forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1$$

$$\forall 1 \leq j \leq m$$

ILP+LP Relaxation+Rounding: Max-SAT

$$\text{maximise } \sum_{j=1}^m z_j$$

subject to

$$\sum_{i: x_i \in C_j} y_i + \sum_{i: \neg x_i \in C_j} (1 - y_i) \geq z_j \quad \forall 1 \leq j \leq m$$

$$0 \leq y_i \leq 1$$

$$\forall 1 \leq i \leq n$$

$$0 \leq z_j \leq 1$$

$$\forall 1 \leq j \leq m$$

Max-SAT: Can we do better?

Max-SAT: Can we do better?

Theorem. The “best-of-two” approach which runs both the naïve randomised algorithm and the randomised rounding gives, in expectation, a $3/4$ -approximation for Max-SAT.

Max-SAT: Can we do better?

Theorem. The “best-of-two” approach which runs both the naïve randomised algorithm and the randomised rounding gives, in expectation, a $3/4$ -approximation for Max-SAT.

Recap

- LPs are **powerful** (but **not enough**)
- We know how to solve them **efficiently** (in theory and practice)
- ILPs are **more powerful**
- We **don't know how to solve them** (though we often can in practice)